California State Polytechnic University, Pomona
College of Engineering
Department of Computer Engineering

Reaction Time Game

ECE 3300 Section 01
Professor Mohamed E E H Aly

May 11, 2026

Group 5
Lupe Becerra
Kayla Tojin
Rodrigo Vallejo
Justin Sov

Table of Contents

Description of the System

The Reaction Time GAme is a real-time digital system implemented on the Nexys A7 FPGA board. The objective of the design is to measure human reaction speed by presenting randomized directional prompts and requiring the user to respond using push-button inputs within a specified timing window. The system integrates multiple hardware modules to form a complete embedded architecture. These include a finite state machine for game control, a VGA display subsystem for graphical output, a UART subsystem for serial logging, and a seven-segment display driver for numerical visualization. Input reliability is ensured through debounce circuits applied to all push-button signals. The system operates in repeating rounds, where each round follows a consistent sequence:

1. The user selects a difficulty mode using the switches.
2. The game begins by toggling SW[0] from high to low, which resets and initializes the system.
3. A random directional target is generated.
4. A randomized delay period occurs (WAIT state).
5. The system enters an active response phase (GO state). The VGA interface displays a graphical button pad representing the Nexys board buttons and highlights a random directional arrow or center button in green, where the player must press the corresponding physical button within the selected time limit. Simultaneously, the seven segment display begins counting upwards in milliseconds to measure the user's reaction time.
6. User input is evaluated for correctness and timing.
7. The system transitions to win, lose, or next round logic.

This structure enables consistent timing evaluation while maintaining unpredictability in user interaction.

Explanation of All Required I/Os

Inputs

CLK100MHZ - Primary 100 MHz system clock from the FPGA board. All synchronous modules operate using this clock.

SW[15:0] - Configuration for mode selection/ reaction time limit selection and reset control.Each switch within each mode decreases the allowed response time by two hundred milliseconds from right to left across the board.

SW[15:11] - Easy Mode (3300ms, 3100ms, 2900ms, 2500ms)

SW[10:6] - Normal Mode (2300ms, 2100ms, 1900ms, 1700ms, 1500ms)

SW[5-1] - Hard Mode (1300ms, 1100ms, 900ms, 700ms, 500ms)

SW[0] - toggled to begin and restart the game.

Push Buttons - Directional user inputs.

BTNU - Up Input
BTND - Down Input
BTNL - Left Input
BTNR - Right Input
BTNC - Center Input

Outputs

LED[15:0] - Real-time visual game state indicators. LED[15:0] turns on when the user wins the game and when the correct button is pressed before the time runs out.

7-Segment Display (SEG, AN, DP) - Displays user reaction time by counting up in milliseconds.

UART_TX - Serial output stream utilizing PuTTY to display mode, time, and result of each round played by the user.

- PASS: the correct button is pressed within the required time, the round is considered successful
- LOSE: the wrong button is pressed, the button is pressed too early, or the response time exceeds the limit, the round is considered a failure.
- WIN: the user successfully pressed the correct button within the required time ten rounds in a row.

VGA Interface (Hsync, Vsync, RGB signals) - Generates graphical user interface that highlights the active target direction, and changes the background color depending on the current gameplay condition.

Green Screen with Button Pad- Waiting State

Displayed during the WAIT_STATE, before the reaction prompt becomes active. This indicates that the player must wait and avoid pressing any buttons prematurely.

Green Screen with Blue Highlighted Direction - GO State

Displayed during the GO_STATE, when the player is expected to respond to the highlighted directional prompt. The active target arrow is visually emphasized on the screen.

Purple Screen - Result State

Displayed when the player successfully completes the required gameplay. The screen includes a “YOU WIN!” message rendered using VGA pixel graphics.

Red Screen - Lose State

Displayed when the player presses the incorrect button, reacts too early, or exceeds the allowed reaction time. The screen shows a “YOU LOSE!!!” message to indicate a failed attempt.

FSM Design

The finite state machine is the core controller of the entire project. It controls gameplay flow, reaction timing, direction generation, win and loss conditions, and game progression. The FSM begins in the IDLE state. In this state all counters and outputs are reset. The system also selects a random direction using an LFSR-based random generator and generates a random wait period before gameplay begins. After initialization the FSM transitions to the WAIT_STATE. During this state the player must wait without pressing any buttons. If the player presses any button early, the FSM immediately transitions into the lose state. Otherwise, the FSM waits until the randomly generated delay expires. Once the wait timer finishes, the FSM enters the GO_STATE. During this phase the VGA module highlights the required direction, and the reaction timer begins counting milliseconds. The player must press the correct button before the reaction time exceeds the selected time limit. If the correct button is pressed, the FSM transitions into the ROUND_WIN state. The system increments the round counter, flashes the LEDs, and triggers UART/FIFO logging. If the player has completed enough successful rounds, the FSM transitions into the final result state and declares a win condition. Otherwise, the FSM enters the NEXT_ROUND state and prepares for another challenge. If the wrong button is pressed or the timer exceeds the allowed limit, the FSM transitions into the LOSE_STATE. The lose state activates a red VGA background and log the failed attempt through UART and FIFO modules. The FSM then transitions into the RESULT state where the final game screen remains displayed until reset. This FSM demonstrates sequential state transitions, timing control, randomization, conditional branching, and synchronous digital system design.

Explanation of FIFO

The FIFO module provides temporary storage for game results. The FIFO in this project has a width of twenty bits and a depth of eight entries. The stored data contains the win flag, lose flag, selected mode, and reaction time. Every time a game round completes, the FSM activates the log_start signal, which causes the current game result to be written into the FIFO. The FIFO maintains separate read and write pointers along with a count register. The write pointer tracks where the next word should be stored while the read pointer tracks which value should be read next. The count register keeps track of how many entries are currently stored. The FIFO also generates full and empty status signals. The full signal becomes active when the FIFO reaches maximum capacity, while the empty signal becomes active when no data is stored. These signals help prevent invalid read or write operations. The FIFO design demonstrates synchronous memory control, pointer arithmetic, and buffered communication between hardware modules.

Explanation of Modeling Styles Used

Behavioral Modeling

Behavioral Modeling was implemented using always blocks to describe the behavior of counters, timers, FSM transitions, UART transmission logic, VGA timing, and debouncing. The most significant use of behavioral modeling appears in the game_fsm module, which implements the finite state machine responsible for controlling gameplay, timing, scoring, and state transitions.

Dataflow Modeling

Dataflow modeling was used through continuous assignments with the assign keyword. This style was used for combinational logic such as button detection, random direction selection, and condition checking. Dataflow modeling allows Boolean equations and combination logic to be expressed directly.

```
assign any_button = btnU | btnD | btnL | btnR;
assign wrong_button = any_button && !correct_button;
```

The seven-segment display driver also uses dataflow modeling to separate the reaction time value into individual decimal digits for display:

```
assign ones = number % 10;
assign tens = (number / 10) % 10;
assign hundreds = (number / 100) % 10;
assign thousands = (number / 1000) % 10;
```

Gate-level Modeling

Gate-level modeling was demonstrated using explicit logic gate instantiations. The any_button logic uses OR gates to combine all button signals into one output. This style represents hardware at the individual gate level and demonstrates structural digital design concepts.

```
or g1(t1, btnU, btnD);
or g2(t2, btnL, btnR);
or g3(t3, t1, t2);
or g4(any_button, t3, btnC);
```

The first two OR gates combine pairs of directional buttons. The third OR gate merges those intermediate signals, while the final gate includes the center button. As a result, the output signal any_button becomes high whenever any push-button on the Nexys board is pressed.

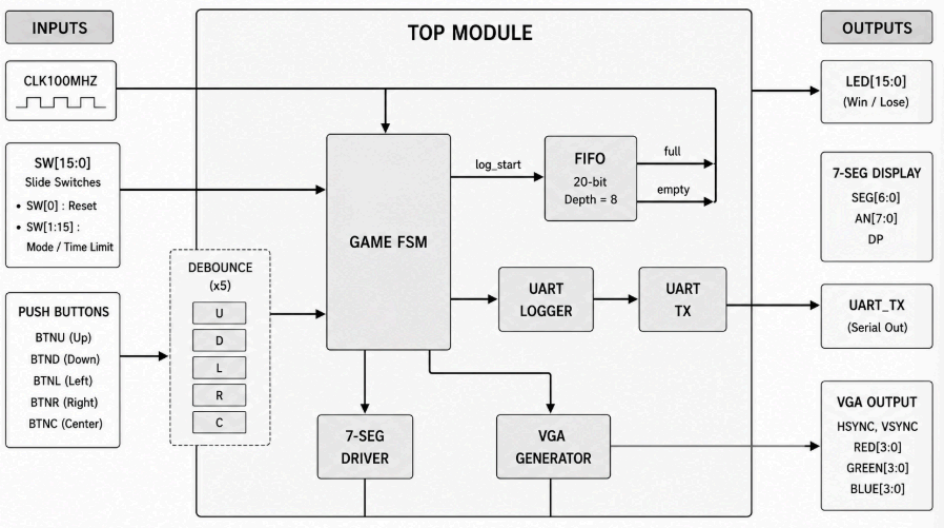
Structural modeling

Structural modeling was used in the top-level module where all submodules were instantiated and connected together. The top module serves as the structural integration layer for the entire project.

Explanation of Testbench Strategy

The mother_ tb module serves as the master verification environment for the project. Instead of testing only one module, the testbench verifies both individual modules and the fully integrated system. The testbench begins by generating a 100 MHz simulation clock. This clock drives all modules during simulation exactly as the FPGA clock would during hardware execution. The verification process is organized using reusable Verilog tasks. Each task performs a specific test operation, making the testbench modular and easier to understand. The FIFO test results reset the FIFO, writes several values into memory, and reads them back out. The simulation checks whether the values emerge in the same order they were written. This confirms the correct FIFO operation. The debounce test applies noisy button transitions and observes the clean debounced output. This verifies that unstable mechanical button signals are filtered properly. The UART test sends an ASCII character and checks whether the UART transmitter becomes busy and produces serial output timing correctly. The top-level game tests verify FSM functionality. The testbench automatically selects different game modes, forces shorter wait times for faster simulation, waits for the FSM to enter the GO state, and then simulates correct or incorrect button presses. The simulation checks whether the FSM responds properly to each condition. This verification strategy demonstrates automated hardware settings, modular testbench design, reusable tas, and full-system validation.

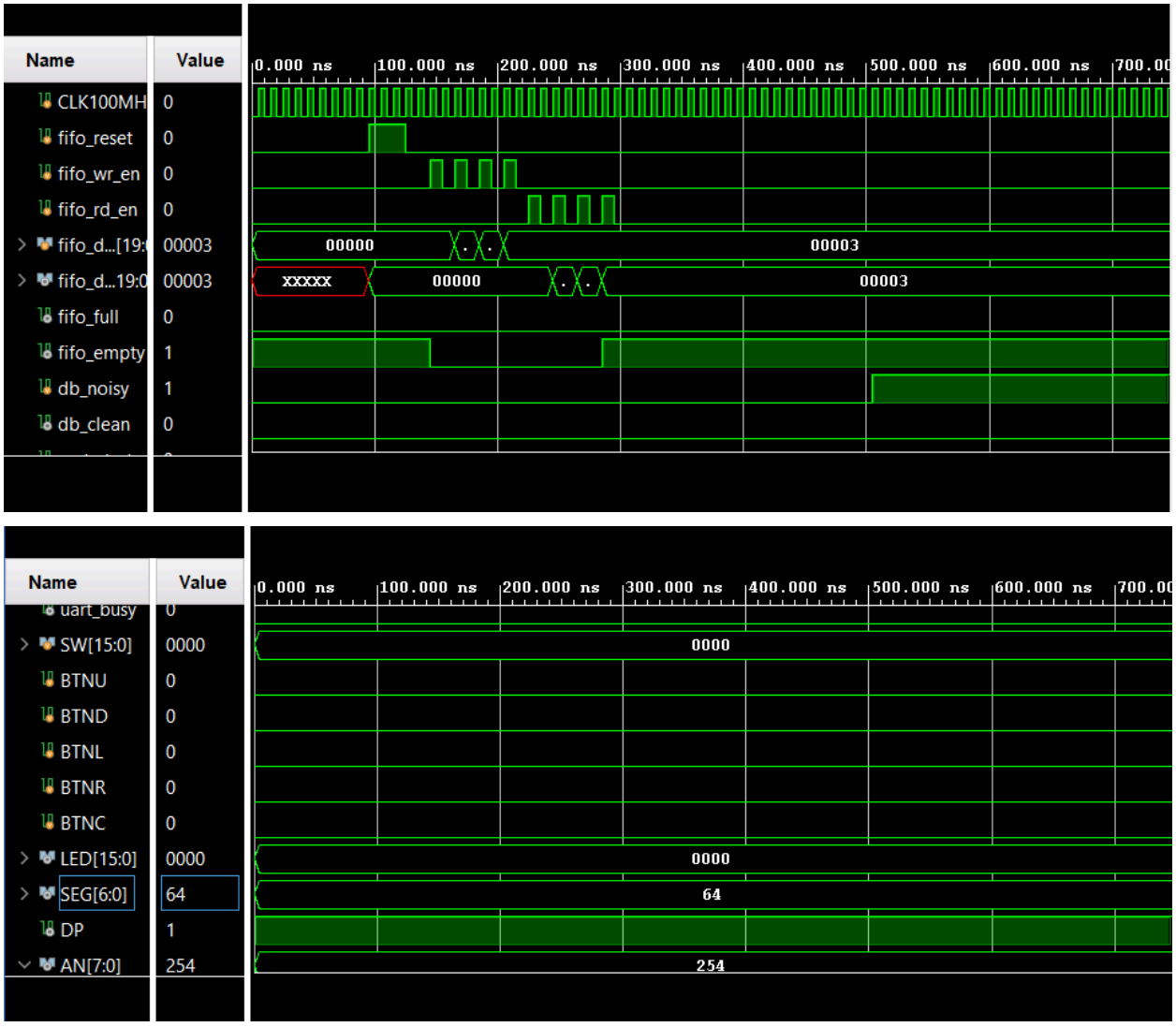
Block Diagram



Simulation Results

Simulation waveforms confirm that all modules operate correctly. The FIFO simulation shows reset behavior, sequential writes, and sequential reads. The waveform confirms that values are stored and retrieved in proper FIFO order. Signals such as fifo_wr_en, fifo_rd_en, fifo_empty, and fifo_dout demonstrate correct memory behavior. The UART waveform shows proper serial communication timing. The transmitter begins in an idle HIGH state, produces a LOW start bit, transmits eight data bits, and ends with a HIGH stop bit. The FSM simulation confirms proper state transitions between idle, waiting, and gameplay, win, and lose conditions.

The waveform also shows reaction timer increments and correct direction matching during gameplay. The VGA module generates correct synchronization signals and the graphical output timing for display operation. The simulation console outputs PASS messages throughout the testbench execution, confirming that the system satisfies all required functionality.



*mother*_tb *waveforms*

Conclusion

This project successfully demonstrates the design and verification of a complete FPGA-based reaction time game using Verilog HDL. The system integrates several important digital design concepts into one functional hardware application, including finite state machines, FIFO memory, VGA graphics, UART communication, debounding, and seven-segment display control. The project also demonstrates the use of multiple Verilog modeling styles, including behavioral, structural, and gate-level modeling. The master testbench provides comprehensive

verification of both individuals and the integrated system. Overall, the project demonstrates strong understanding of FPGA system integration, synchronous digital design, hardware verification, and real-time user interaction using Verilog HDL.